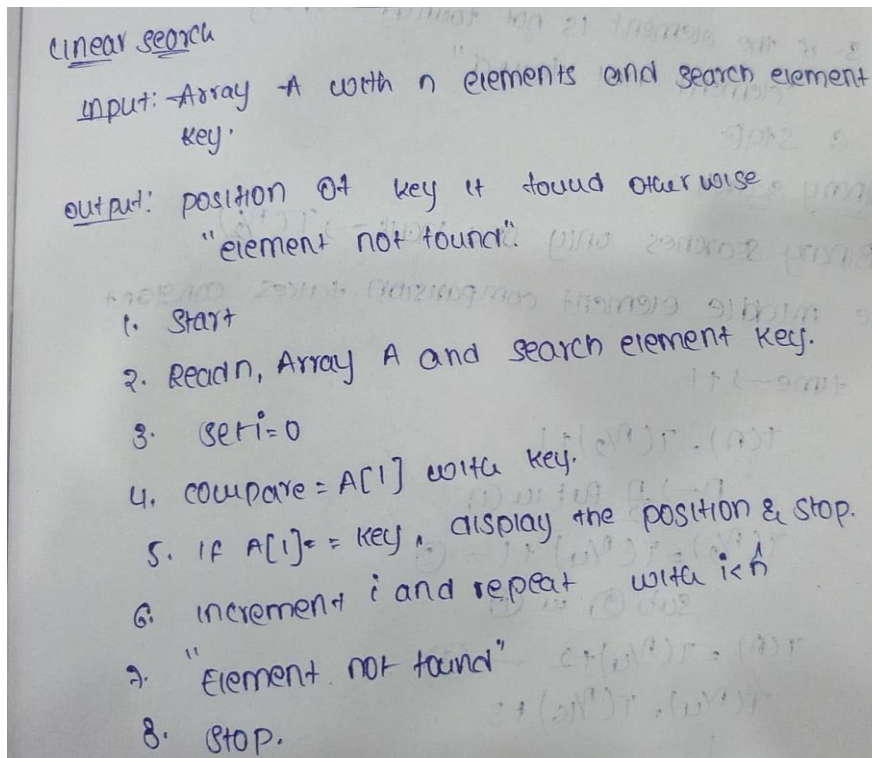


PRACTICAL: 02

Aim: To implement and find the time analysis of Linear Search and Binary Search algorithm.

A) Linear Search:

Algorithm:



Program / Code:

```
#include <iostream>

using namespace std;

int main() {
    int arr[100],i,n,key;

    cout<<"enter number of array elements:"<<endl;

    cin>>n;

    cout<<"enter array elements:"<<endl;

    for(i=0;i<n;i++){
        cin>>arr[i];
    }

    cout<<"enter element to serach:"<<endl;

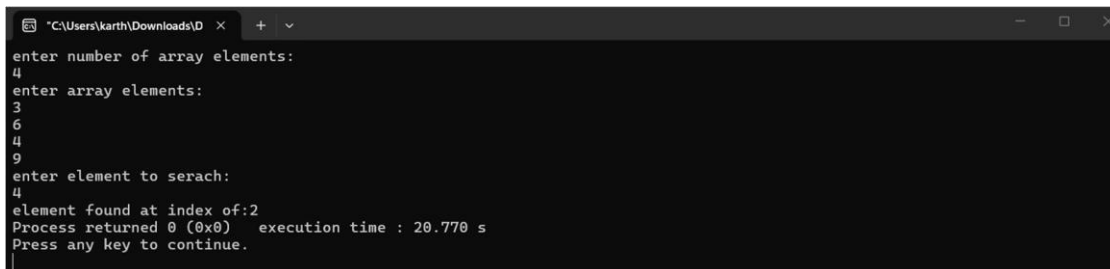
    cin>>key;

    for(i=0;i<n;i++){
        if(arr[i]==key)
        {
            cout<<"element found at index of:"<<i;

        }
    }

    return 0;
}
```

Output:



```
"C:\Users\karth\Downloads\ID"
enter number of array elements:
4
enter array elements:
3
6
4
9
enter element to serach:
4
element found at index of:2
Process returned 0 (0x0) execution time : 20.770 s
Press any key to continue.
```

Time Complexity:

Time complexity:

```

for (i=0; i<n; i++) {
    if (A[i] == key) {
        cout << "Element found";
        break;
    }
}

```

$T(n) = 1 + 1 + 1 + n$
 $T(n) = O(n)$
 in worst case
 Best case $\rightarrow O(1)$
 Average case $\rightarrow O(1)$

Linear search:

i) Best case: $O(1)$
 ex: arr = {10, 20, 30, 40, 50}
 key = 10 There is only time compare
 $T(n) = 1$
 Time complexity = $O(1)$

ii) Average case: $O(n)$
 element found at middle
 ex: arr = {10, 20, 30, 40, 50}
 key = 30 $\rightarrow \frac{n}{2}$ compare
 $\frac{4}{2} = 2$ arr[1], arr[2] = 30
 $T(n) = \frac{n}{2}$
 $T(n) = O(n)$

iii) worst case: $O(n)$
 element found at last
 ex: arr = {10, 20, 30, 40, 50}
 key = 50
 $T(n) = n$
 $T(n) = O(n)$

Time complexity linear search is $O(n)$

B) Binary Search:

Algorithm:

Binary search

Input: Sorted Array A with n elements and search element, key.

Output: position of key if found, otherwise "element not found"

1. Start
2. Read n, sorted array A and search element key
3. Set low=0 and high = n-1
4. Repeat while low ≤ high
 - set mid = (low + high) / 2
 - If A[mid] == key, display the position and stop
 - If key < A[mid], set high = mid - 1
 - Otherwise, set low = mid + 1
5. If the element is not found, display "element not found"
6. Stop

Program / Code:

```
#include <iostream>

using namespace std;

void bubblesort(int arr[], int n) {
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(arr[j] > arr[j+1]) {
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}

int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n-1;
    while(low <= high) {
        int mid = (low + high) / 2;
        if(arr[mid] == key) return mid;
        else if(arr[mid] < key) low = mid+1;
        else high = mid-1;
    }
    return -1;
}

int main() {
    int arr[100], n, key;
    cout << "Enter number of array elements: ";
```

```
cin >> n;

cout << "Enter array elements: ";

for(int i=0; i<n; i++) {
    cin >> arr[i];
}

bubblesort(arr, n);

cout << "Sorted elements: ";

for(int i=0; i<n; i++) {
    cout << arr[i] << " ";
}

cout << endl;

cout << "Enter element to search: ";

cin >> key;

int result = binarySearch(arr, n, key);

if(result != -1)

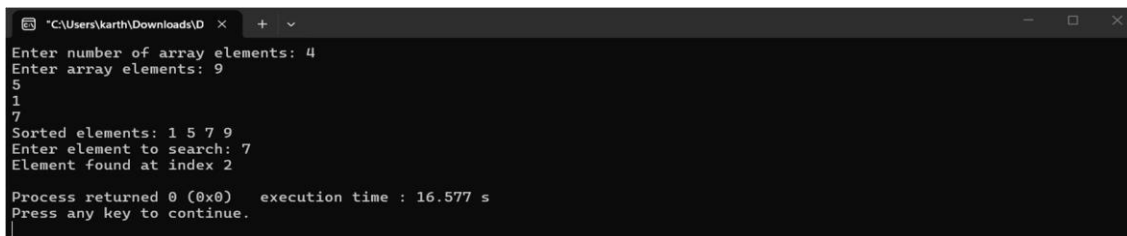
    cout << "Element found at index " << result << endl;

else

    cout << "Element not found" << endl;

return 0;
}
```

Output:



```
"C:\Users\karth\Downloads\ID" x + v
Enter number of array elements: 4
Enter array elements: 9
5
1
7
Sorted elements: 1 5 7 9
Enter element to search: 7
Element found at index 2

Process returned 0 (0x0)   execution time : 16.577 s
Press any key to continue.
```

Time Complexity:

Time complexity

binary search (A[]; n; key) {

low = 0;

high = n - 1;

while (low <= high) {

mid = (low + high) / 2;

if (A[mid] == key) {

return mid;

else if (key < A[mid]) {

high = mid - 1;

else {

low = mid + 1;

}

}

return -1;

$T(n) = T(n/2) + 1$

$T(n) = \begin{cases} 1 & n=1 \\ T(n/2) + 1 & n>1 \end{cases}$

$T(n) = T(n/2) + 1$

$n \rightarrow n/2$ $T(n/2) = T(n/4) + 1$

$= T(n/4) + 2$

$n \rightarrow n/4$

$T(n) = T(n/4) + 2$

$= T(n/8) + 2 + 1$

$= T(n/8) + 3$

$T(n) = T(n/8) + 3$

$= T(n/8^k) + k \rightarrow T(n/n) + \log_2 n$

$T(1) + \log_2 n$

$= 1 + \log_2 n$

$T(n) = O(\log n) \rightarrow$ in worst case

$O(1) \rightarrow$ in best case

$O(\log n) \rightarrow$ in average case

$n=1$

$\frac{n}{2^k} = 1$

$n = 2^k$

$\log n = \log 2^k$

$\log n = k \log 2$

$k = \frac{\log n}{\log 2}$

$k = \log_2 n$

Binary search:

i) Best case:

$= O(1)$

$mid = (low + high) / 2$

if (A[mid] == key $\rightarrow$ 1

$T(n) = 1 + 1 = 2$

$T(n) = O(1)$

iii) Worst case:

$n \rightarrow \frac{n}{2} \rightarrow \frac{n}{4} \rightarrow \frac{n}{8} \dots$

$T(n) = \log_2 n$

$T(n) = O(\log n)$

(ii) Average case: $O(\log n)$

$n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8$

no. of divisions $\log_2 n$

$T(n) = \log_2 n$

$T(n) = O(\log n)$

Conclusion:

In this practical, we studied and implemented two searching algorithms: Linear Search checks each element one by one, so it takes more time as the list size grows. Binary Search is much faster because it works only on sorted lists and cuts the list in half every time, making it a better choice for large data.